MQTT-Discovery and UPnP: a comparison

Daniele Foschi
Alma Mater Studiorum Università di Bologna UNIBO
Bologna, Italy
daniele.foschi2@studio.unibo.it
https://github.com/DaniDF/MQTT-Discovery-vs-UPnP

***Abstract*—TODO**

***Index Terms*—MQTT, discovery, upnp, project activity.**

I. Introduction

Internet of Things (IoT) has a lot of definitions: from International Telecommunication Union is *"A global infrastructure for the information society, enabling advanced services by interconnecting (physical and virtual) things based on existing and evolving interoperable information and communication technologies"*. IEEE defines IoT as a list of features:

- A network
- Objects are identifiable as things with unique identifiers
- When a thing communicate should use internet at least in one segment of the communication
- Things can have sensing or actuating or both capabilities
- They should be programmable as result of injection of intelligence
- The offered services should be available anywhere, anytime and for anything

In a environment populated by a large variety of different devices with different needs and capabilities designing a standard useful for communicating with them is a challenge. Discovery protocols are designed to reconcile the dichotomy between the requirement for highly specific, ad-hoc protocols capable of granular device control and the necessity for generalised frameworks that can accommodate a highly heterogeneous ecosystem of devices. By employing these protocols, an IoT device achieves autonomous configuration through active participation in the advertisement and solicitation of network-based services. Devices functioning as service providers systematically broadcast their available resources, the specific services they offer, and the operational guidelines for invocation, such as the appropriate Application Programming Interfaces (APIs). Furthermore, discovery protocols streamline the connectivity paradigm between clients and target resources or services. Consequently, this architecture ensures that clients are adequately equipped to transmit necessary input parameters to the services and successfully retrieve the corresponding computational results.

In particular this dissertation is focused on Universal Plug and Play (UPnP) and a protocol based on Message Queuing Telemetry Transport (MQTT) knwon as MQTT-Discovery (MQTT-D). UPnP is a discovery protocol without infrastructural nodes where each node can advertise and listen for services. In particular nodes that make available services are called *devices* and those searching for services are called *control-points*. In MQTT-D there is a third entity called *broker* that facilitates the communication between services and clients.

This dissertation wants to highlight the performance differences between UPnP and MQTT-D in IoT environment. The main focus is measuring and comparing latency and robustness in different deployment configurations: how different amount of data producers and consumers affect the protocol performances.

TODO frase sui risultati
TODO add remainder of

II. Discovery protocols for IoT

Resource discovery protocols can be categorised into: centralised and distributed. In centralised approaches such as MQTT, resources are registered in a directory that serves as intermediary between providers and users. In distributed approaches such as UPnP, users discover active resources through direct communication with resources in a Peer-to-Peer (P2P) fashion with no need for an intermediary directory [1].

UPnP is a P2P discovery service without hierarchical structuring. In UPnP two entities can be distinguished: *control-point* the entity that look for services and resources and *device* the entity that make services available. A device can dynamically join a network and obtain a locally valid *IP* address. After it can make its capabilities visible and discover others, if present, and dynamically leave the network. UPnP uses *XML* for the description of the resources, services and action invocations.

UPnP can be divided into 4 parts: Simple Service Discovery Protocol (SSDP), HyperText Transfer Protocol (HTTP), Simple Object Access Protocol (SOAP) and General Event Notification Architecture (GENA). SSDP is a network protocol based on the Internet protocol suite for advertisement and discovery of network services and presence information. It is a text-based protocol based on a non fully compliant version of HTTP over UDP [2].

In SSDP, whenever a node intends to discover a *device*, it issues an M-SEARCH request specifying the desired device

<table>
<tr><td colspan="4">UPnP vendor [purple-italic]</td></tr>
<tr><td colspan="4">UPnP Forum [red-italic]</td></tr>
<tr><td colspan="4">UPnP Device Architecture [green-bold]</td></tr>
<tr><td rowspan="2">SSDP [blue]</td><td rowspan="2">Multicast events [navy-bold]</td><td>SOAP [blue]</td><td>GENA [navy-bold]</td></tr>
<tr><td>HTTP [black]</td><td>HTTP [black]</td></tr>
<tr><td colspan="2">UDP [black]</td><td colspan="2">TCP [black]</td></tr>
<tr><td colspan="4">IP [black]</td></tr>
</table>

Fig. 1. UPnP Protocol Stack [2]

characteristics. Upon receiving this request, a *device* determines whether it satisfies the specified criteria and, if so, returns an appropriate response. Since the number of potential responses cannot be known in advance, a burst of simultaneous replies on the network may lead to overloading of the *control point*. To mitigate this risk, the *control point* includes the MX parameter in the search request [2]. This parameter defines the maximum number of seconds within which responding *devices* may transmit their replies. Consequently, each *device* waits for a random interval between 0 and the value specified by MX before sending its response.

SOAP is a messaging protocol specification for exchanging structured information in XML message format. It is used by the *control-point* for requesting the execution of services with specified arguments on the *device* and by the *device* for returning the results. GENA is an event notification protocol used by *devices* to publish variables updates to subscribed *control-points*. GENA supports both unicast and multicast event notification.

MQTT-D is a discovery protocol based on MQTT. MQTT is a lightweight publish-subscribe messaging protocol designed for constrained devices. MQTT-D is a metadata-exchange convention built on top of the MQTT protocol that standardises how devices and services describe themselves and publish their configuration to an MQTT broker. It enables automatic detection, registration, and integration of devices in home-automation or IoT ecosystems without requiring manual configuration.

The MQTT protocol assigns a specific Quality Of Service (QOS) level to each message, establishing the delivery guarantees required across potentially unreliable networks. The protocol defines three hierarchical levels of service: QOS 0 (At-Most-Once), which provides best-effort delivery without retransmission mechanisms; QOS 1 (At-Least-Once), which ensures delivery but permits the possibility of duplicate messages; and QOS 2 (Exactly-Once), the most complex level, which employs a multi-step handshake to guarantee that a message is delivered once and only once. Furthermore, publishers can append a *retained* flag to a message. This instructs the MQTT broker to persist the last message for a given topic, ensuring that any newly subscribed clients immediately receive the most recent data upon connection, independent of the initial publication event.

This dissertation aims to evaluate the performance of MQTT-D and UPnP in deployments with different number of deployed devices and different number of control devices. For MQTT-D will be evaluated how the performances changes with respect to the requested QOS. The evaluated metrics for this study are latency and robustness. The first wants to measure the time needed for discovery and the execution of the first interaction between *control-point* and *device*. The second wants to measure the ratio between the discovered devices and the actual deployed devices. TODO Da vedere magari da girare perché i grafici mostrano l'errore quindi la percentuale dei mancanti non dei presenti

III. Implementation

To establish a testing environment, a fully compatible UPnP *device* and *control point* were implemented in accordance with the official specification [2]. All major protocol components were developed, including SSDP, SOAP, and GENA [3]. For HTTP, the standard library provided by Go was employed. To further validate standards compliance and interoperability, a third-party UPnP client library was also used [4].

For MQTT-D, an existing implementation of the MQTT protocol was adopted, while only a wrapper for the MQTT-D data format was developed. As the MQTT broker, Mosquitto was selected due to its widespread adoption and popularity in IoT deployments.

MQTT-D defines two mechanisms for *device* presence discovery. The first relies on a retained message published by a *device* when it joins the network; this message is subsequently delivered by the broker to a *control point* when the latter subscribes to the corresponding topic [5]. The second mechanism is based on solicitation: the *control-point* publishes a message on a predetermined topic to which all *devices* are subscribed. Upon receiving such a solicitation message, each *device* responds by sending its discovery message [5].

The software developed for this work [3] supports both mechanisms; however, the experiments were conducted exclusively using the solicitation-based approach.

All software components developed by the author [3] were implemented in Go in order to balance high execution efficiency with the advantages of a higher-level programming language.

IV. Experimental results

Experimental evaluations were executed in a closed, dedicated network infrastructure to isolate the testing equipment. All trials were performed over both wired and wireless (IEEE 802.11n) connections. The deployment of *devices* and *control-points* was scaled incrementally: initially in unit steps from 1 to 10, and subsequently in decadal steps up to 100. For MQTT, each configuration was evaluated across QOS levels 0, 1, and 2. To mitigate statistical variance, each experimental combination was repeated 30 times, and the results were averaged.

The experiments tried to evaluate the effectiveness of the MX parameter by removing the random wait time from the *device* response.

The following subsections present the measured metrics for values 1, 5, 10, 50, 100 but a more detailed version can be found in appendix A.

A. Latency comparison

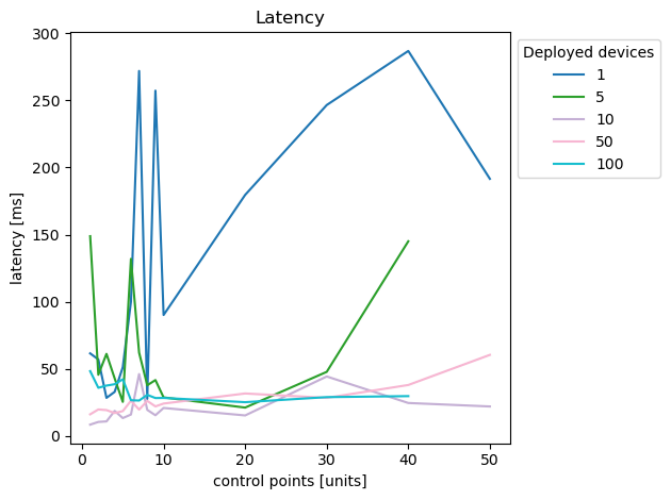

Fig. 2. UPnP latency comparison with different *devices* and *control-points*. See 13.

The comparison shown in Figure 2 reveals a clearly distinguishable performance difference between the configuration with a single deployed *device* and those with higher numbers of deployed *devices*. Furthermore, Figure 13a in Appendix A shows that the difference between the configurations with 2 and 3 deployed devices further supports the observation that setups with fewer deployed *devices* are associated with higher latency. By contrast, configurations involving more than 10 devices yield relatively stable latency values, which begin to diverge noticeably only when the number of *control points* exceeds 30.

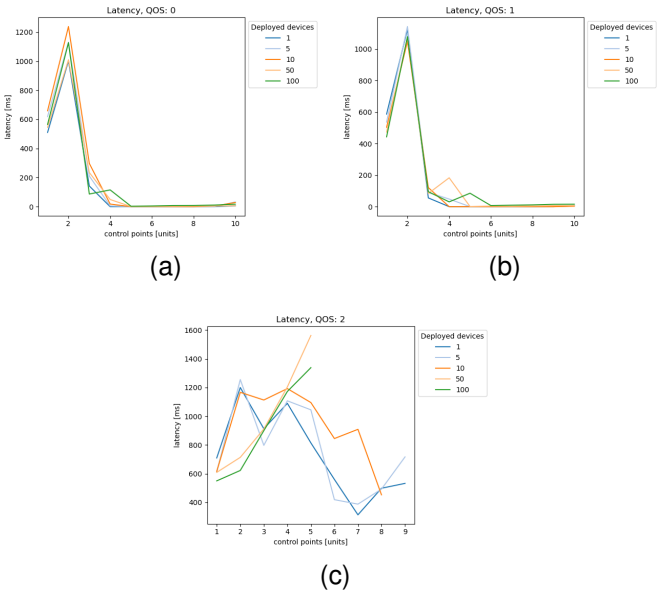

Fig. 3. MQTT latency comparison with different *devices control-points* and QOS. See 7a, 8a, 9a.

The latency measurements obtained from the MQTT-D tests indicate a clear distinction between the lower QOS levels, QOS 0 and 1, and the highest level, QOS 2. As shown in Figures 3a and 3b, configurations with the minimum number of deployed *control-points* exhibit a latency of approximately 600 ms. This value rises to around 1200 ms in the case of 2 deployed *control-points*, after which a sharp decrease is observed. By contrast, this behaviour is not evident in Figure 3c, where the latency values corresponding to 1 and 2 deployed *control-points* remain similar, and the subsequent reduction does not occur at the same configuration point. Instead, elevated latency values persist up to at least 5 deployed *control points*.

1) Comparing the results: The comparison between Figure 2 and Figure 3 indicates that UPnP consistently exhibits lower latency than MQTT-D. For configurations with more than one deployed *control point*, the observed latency values are approximately eight times lower than those of the best-performing MQTT-D configuration.

B. Robustness comparison

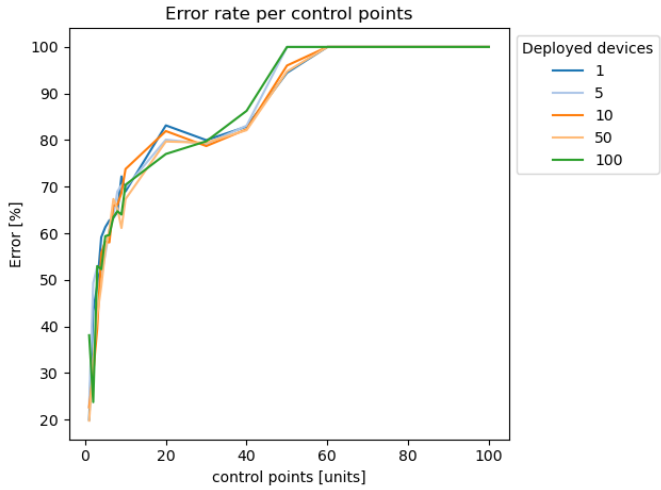

Fig. 4. UPnP robustness comparison with different *devices* and *control-points*. See 15.

For UPnP, the ratio between the number of *devices* discovered by the *control points* and the actual number of deployed *devices* is observed to be consistent across the different configurations (Figure 4). The error rate increases rapidly, following an approximately logarithmic trend up to 20 deployed *control points*; it then remains almost constant until 40 *control points*, after which it rises sharply toward 100%. A more detailed representation in Figure 15 further confirms that this behaviour is nearly identical for all considered configurations.

The results obtained for MQTT-D show that the error rate rises sharply from almost 0% to 100% when the number of deployed *control-points* approaches 10 for every configuration. This suggests that, during the 10-second interval reserved for the discovery phase, none of the *control points* was able to receive the discovery announcement and respond within the required time frame.

1) Comparing the results: A comparison of the robustness of the two discovery protocols indicates that MQTT-D exhibits greater robustness when only a very small number of

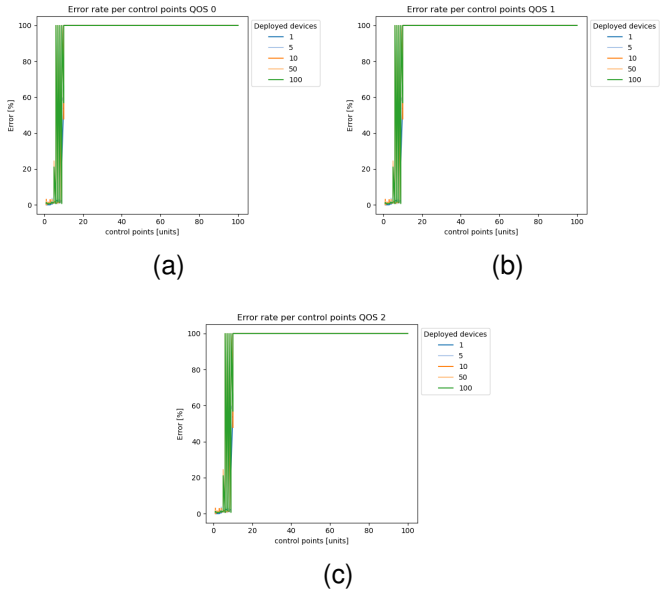

Fig. 5. MQTT robustness comparison with different *devices control-points* and QOS. See 10a, 11a, 12a.

control points is deployed. However, once the threshold of 10 deployed *control points* is reached, MQTT-D displays an error rate of 100%, whereas UPnP still maintains a lower, although substantial, error rate corresponding to approximately 60% of *devices* not being discovered.

C. UPnP MX = 0

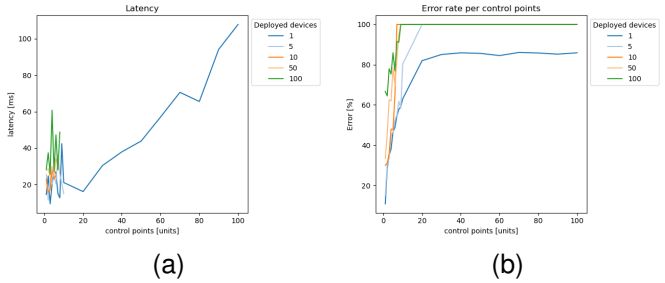

Fig. 6. UPnP (a) latency and (b) robustness different *devices control-points*. See 14, 16.

This experiment was performed by eliminating any artificial waiting time from the UPnP discovery process. A comparison between Figure 6a and Figure IV-A shows that the latency is reduced for all *device* configurations, with the most pronounced improvement occurring in scenarios with a smaller number of deployed *control points*.

In terms of robustness, Figure 6b indicates that, relative to Figure IV-B, the error rate rises more rapidly toward 100%. Nevertheless, the configuration with a single deployed *device* is able to complete the experiment without reaching a 100% error rate, thus exhibiting behaviour different from that shown in Figure IV-B.

D. GENA

TODO Grafici da generare

E. Wired vs Cabled

- Motivazione dei test

F. General comments?????

TODO da capire il nome

V. Conclusions

- Cosa è statto fatto in breve
- Cosa è stato dimostrato
- Future directions

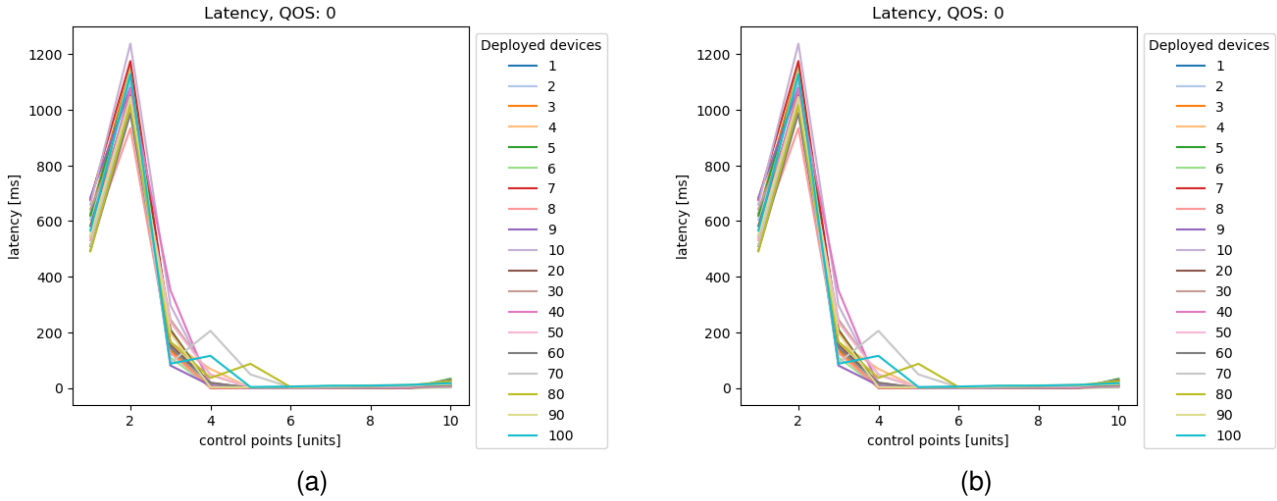

Fig. 7. MQTT latency QOS 0

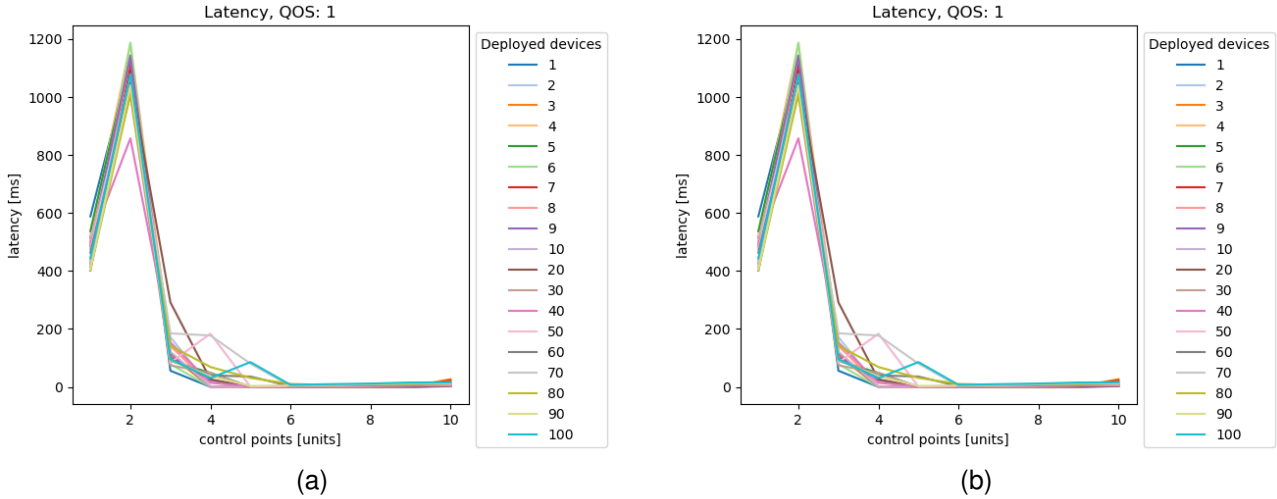

Fig. 8. MQTT latency QOS 1

Appendix

A. MQTT

1) MQTT latency:

2) MQTT robustness:

B. UPnP

1) UPnP latency:

2) UPnP robustness:

References

[1] K. Khalil, K. Elgazzar, and M. Bayoumi, "A comparative analysis on resource discovery protocols for the internet of things," in *2018 IEEE Global Communications Conference (GLOBECOM)*, 2018, pp. 1–7.

[2] O. C. Foundation, "Upnp device architecture 2.0," Open Connectivity Foundation, Tech. Rep., April 2020, document Revision Date: April 17, 2020. [Online]. Available: https://openconnectivity.org/upnp-specs/UPnP-arch-DeviceArchitecture-v2.0-20200417.pdf

[3] DaniDF, "MQTT-Discovery vs UPnP," https://github.com/DaniDF/MQTT-Discovery-vs-UPnP, 2026.

[4] Huin, "goupnp," https://github.com/huin/goupnp, 2026.

[5] O. H. Foundation, "Mqtt discovery," https://www.home-assistant.io/integrations/mqtt/#mqtt-discovery, 2026.

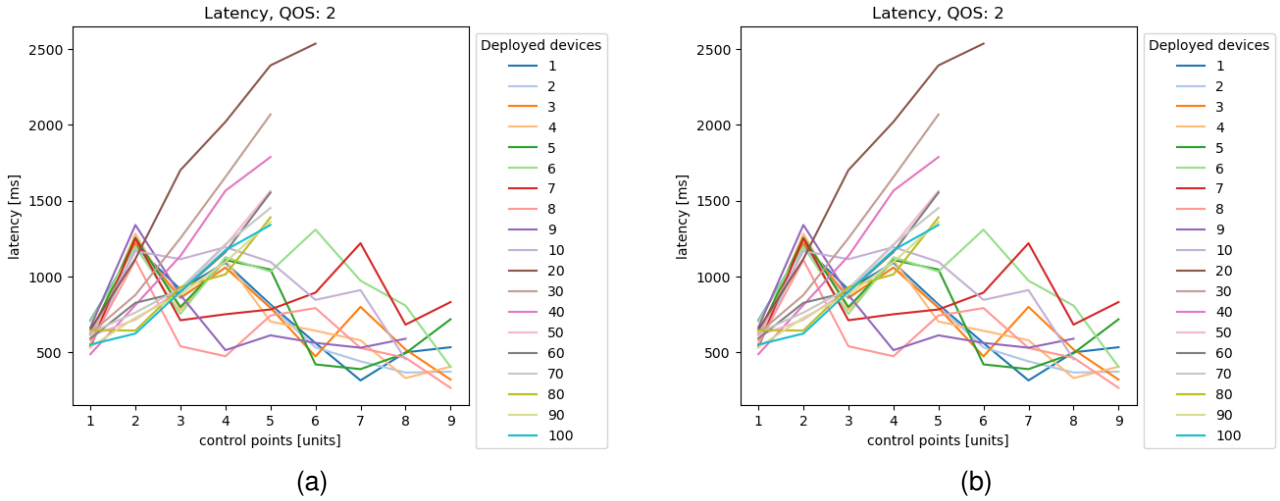

Fig. 9. MQTT latency QOS 2

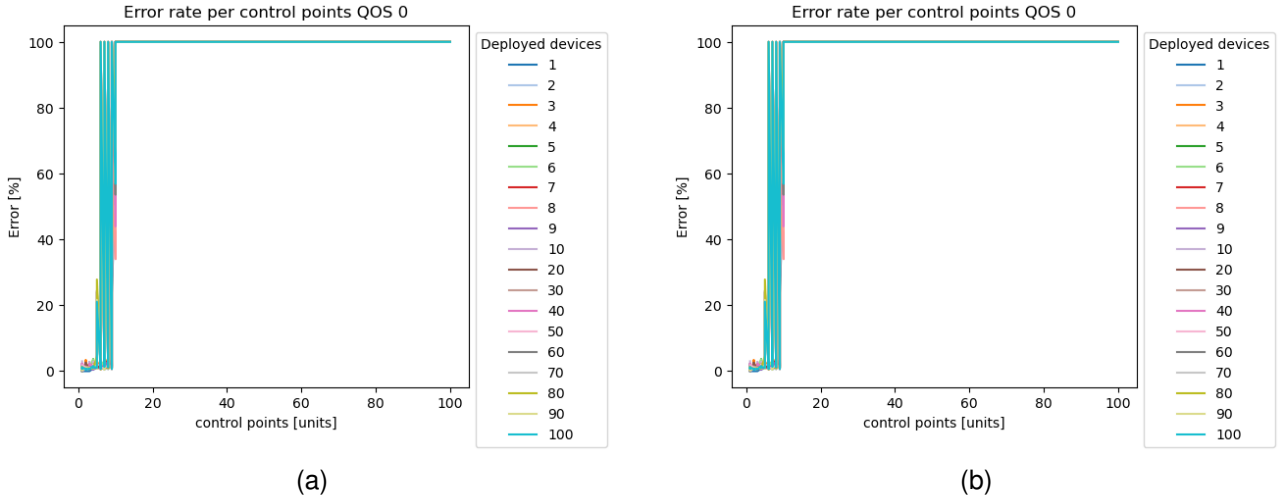

Fig. 10. MQTT latency QOS 0

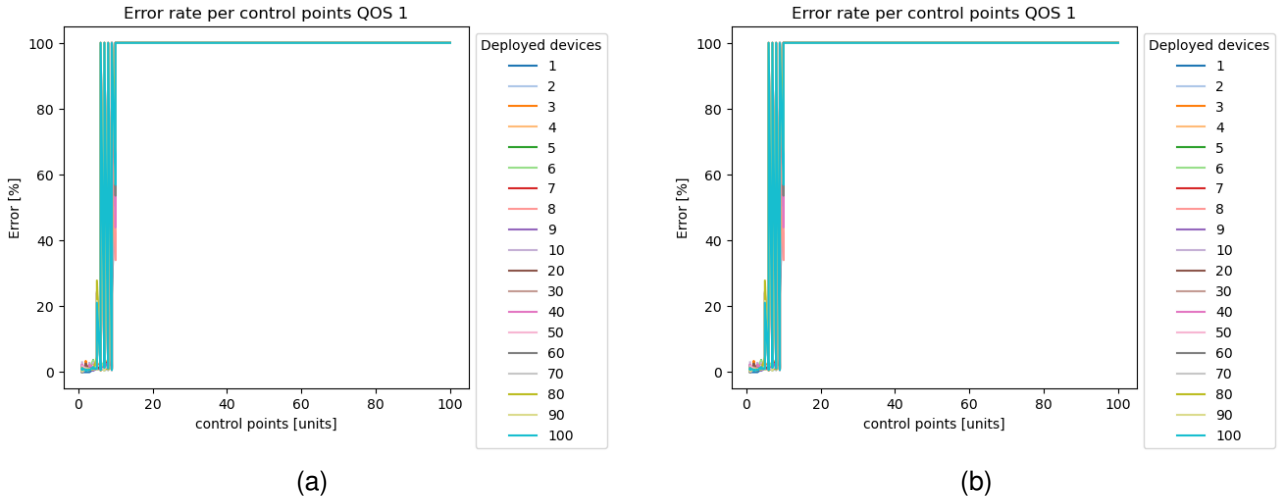

Fig. 11. MQTT latency QOS 1

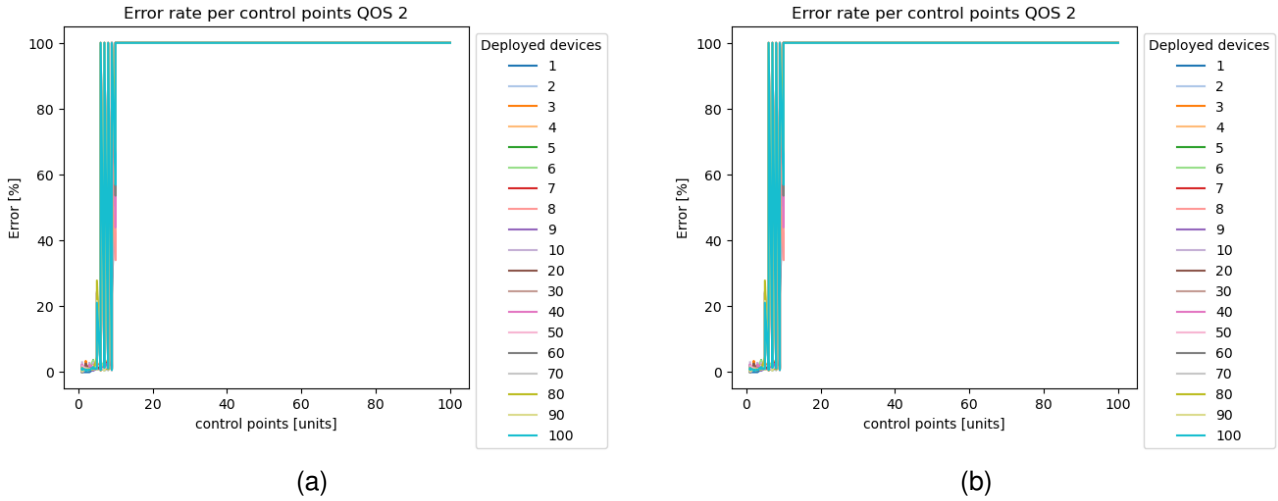

Fig. 12. MQTT latency QOS 2

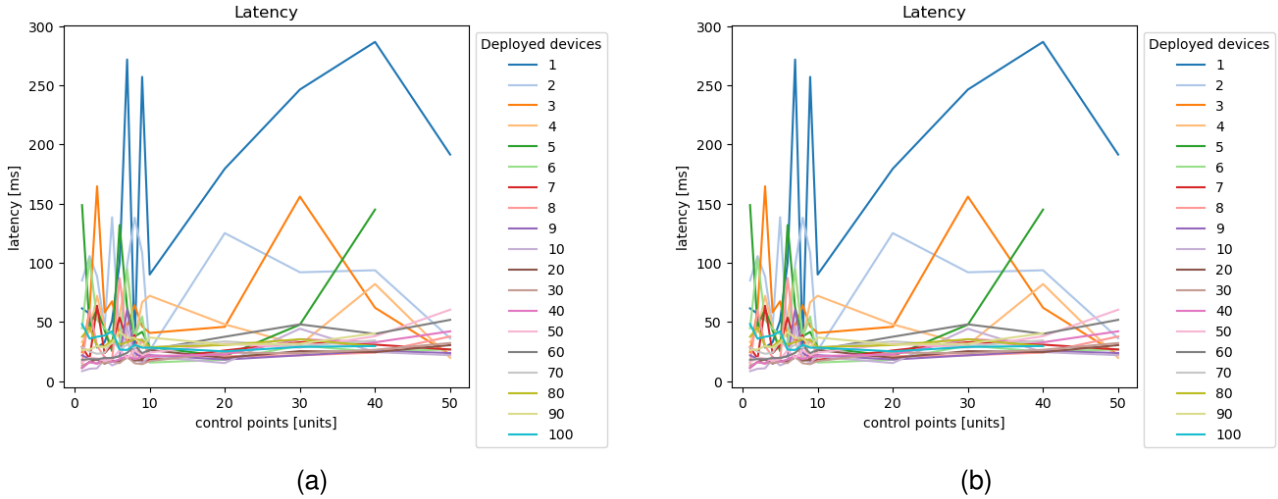

Fig. 13. UPnP latency

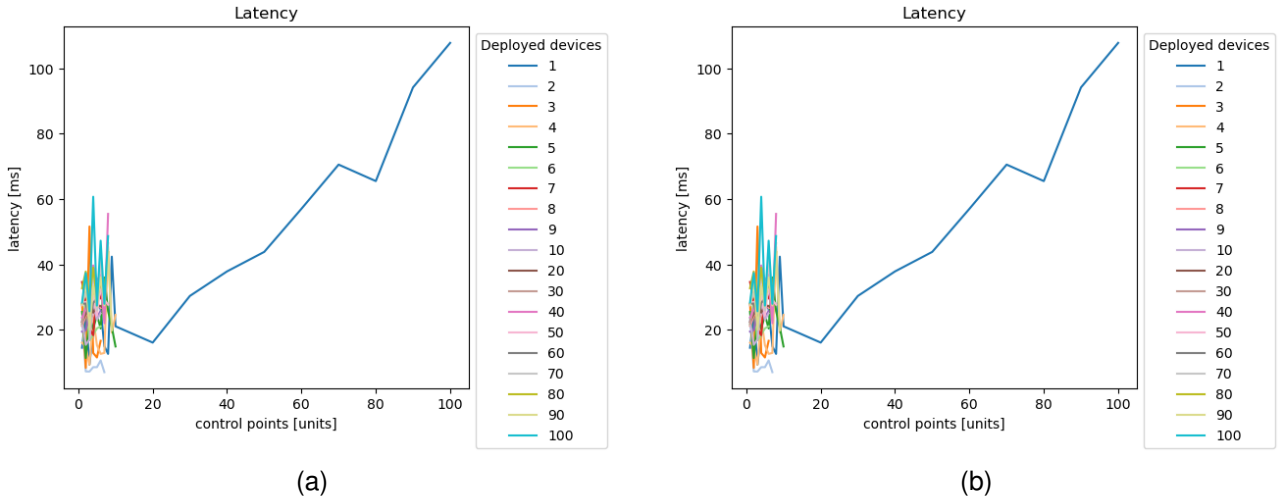

Fig. 14. UPnP latency with no random delay

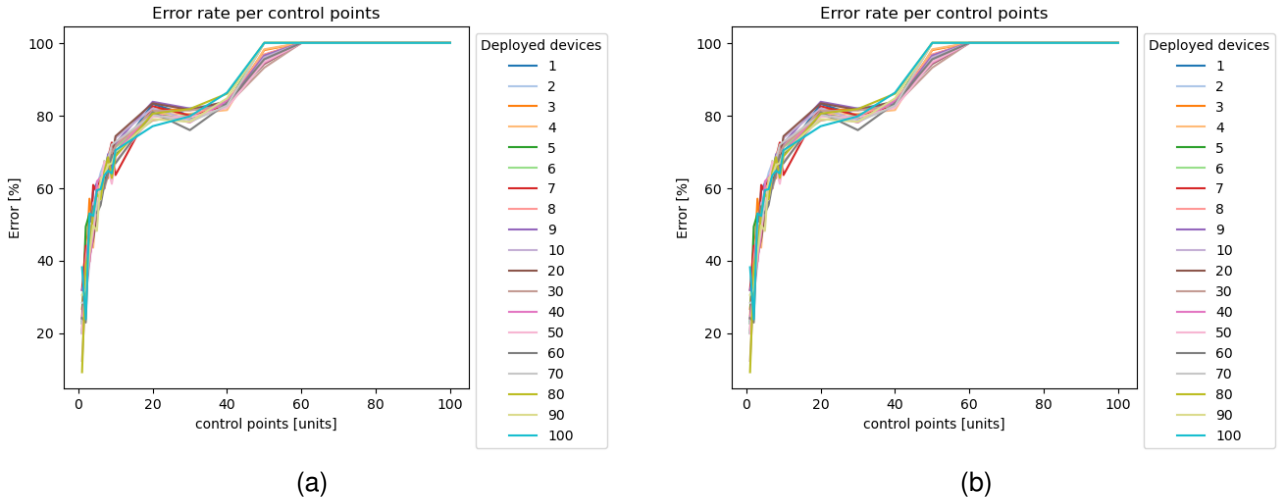

Fig. 15. UPnP robustness

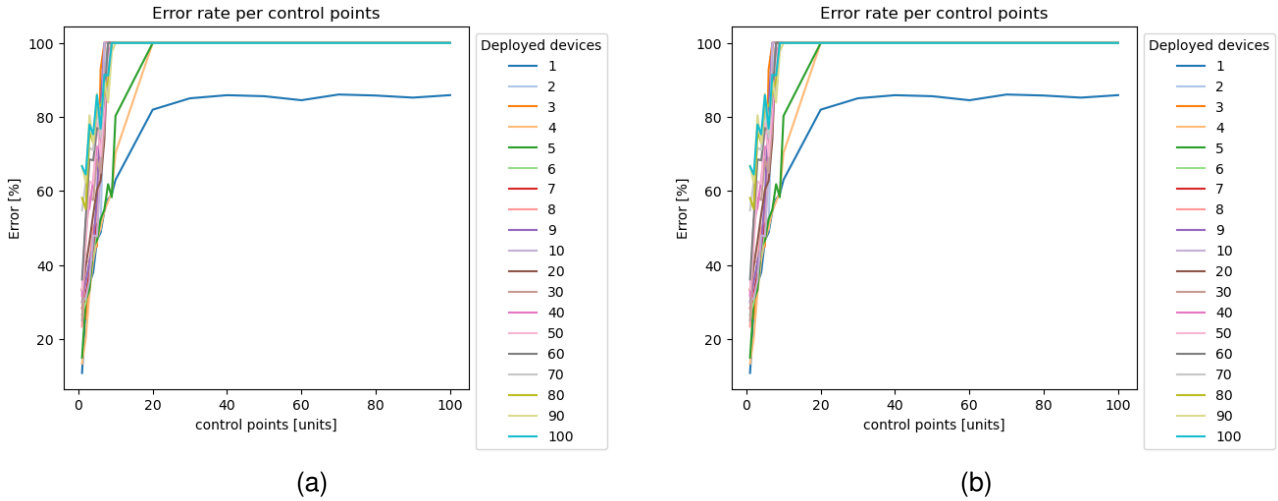

Fig. 16. UPnP robustness with no random delay